Atrium: data model and rendering pipeline

How a website is represented, stored, validated, published and turned into HTML: the data structures, the parsing at every boundary and the caching that makes it fast.

Technical reference · generated from the source in this repository (packages `schema`, `builder-core`, `site-store`, `component-registry`, `renderer-core`, `design-tokens`; apps `api`, `renderer`, `admin`).

1. System overview
2. The data model
3. The node tree
4. Styles, breakpoints and theme tokens
5. The component registry
6. Storage in PostgreSQL
7. Parsing and validation
8. Editing: operations and revisions
9. Publishing: freezing a snapshot
10. Serving a request
11. Rendering a page
12. In the browser
13. Site bundles (export / import)
14. Security at the boundaries
15. Source map

1. System overview

Atrium separates **writing** from **reading**. The admin and visual editor only ever change *drafts* through the API. Publishing freezes the drafts into one immutable JSON document, the *snapshot*. The renderer, a separate Next.js app, only reads published snapshots. It never touches drafts or the database directly.

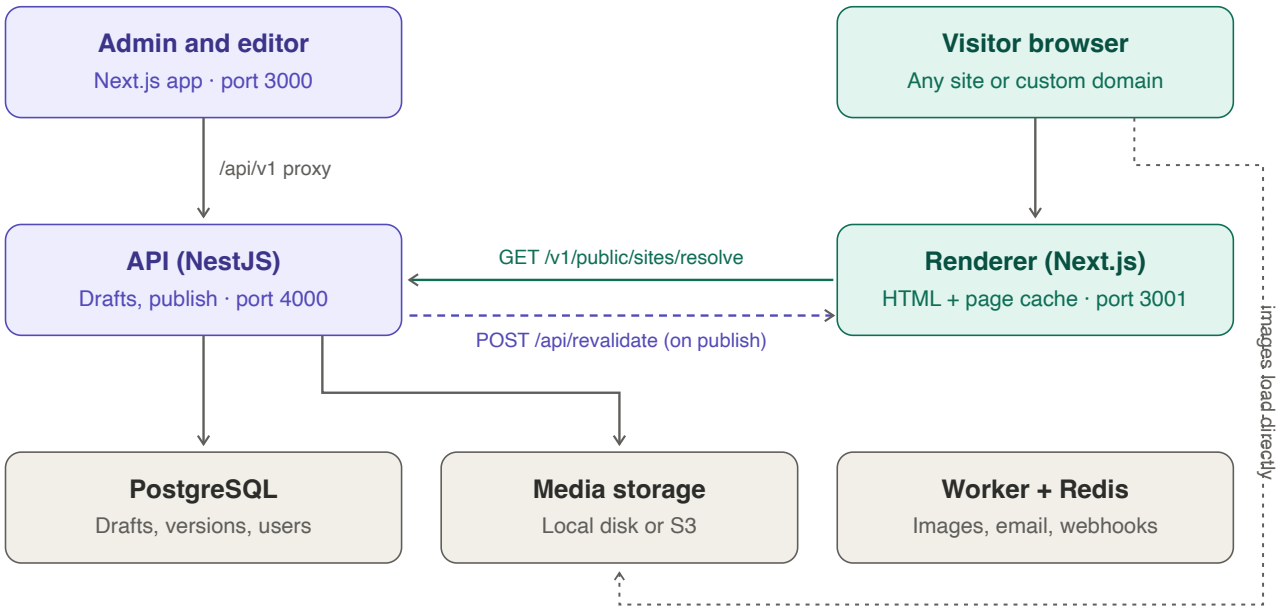


Figure 1. Purple: the write side (drafts). Teal: the read side (published sites). Media files are served straight from storage (`/v1/storage` on the API, or the S3 public URL), not through the renderer.

- **Admin** talks to the API through its same-origin proxy `/api/v1/*`, so the httpOnly refresh cookie works. Uploads stream through that proxy (limit raised to 310 MB).
- **API** owns all writes: drafts, publishing, users, media. It is the only service with full database access.

- **Renderer** resolves a host to a published snapshot through one endpoint, authenticated with the shared `RENDERER_SECRET`, and caches the HTML it renders.
- **Worker** runs queued jobs (BullMQ on Redis): image variants, email, webhooks, scheduled publishing and deployments.

2. The data model

A website is one JSON-serialisable value, `WebsiteDefinition`, defined with Zod in `packages/schema`. The same schema is used by the editor, the API, the AI agent, import/export and the renderer, so every layer agrees on the shape. The renderer receives it wrapped in a `SiteSnapshot`.

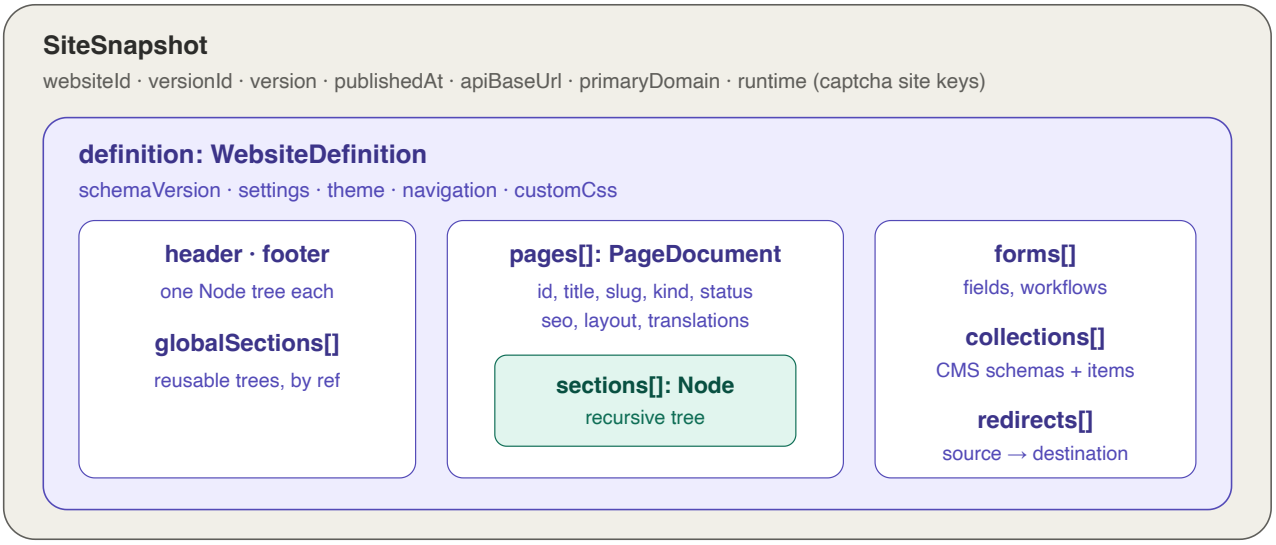


Figure 2. The snapshot envelope and the website definition inside it.

Field	Type	Purpose
`schemaVersion`	literal	Format version; lets old snapshots be migrated.
`settings`	`SiteSettings`	Name, business info (NAP), social links, logo/favicon, site SEO, localization (default locale, locales, URL strategy), scripts, cookie consent, analytics IDs, `performance` (page caching, loading bar), variables, 404 page.
`theme`	`ThemeTokens`	Design tokens: colours, typography, spacing, radius, shadows, borders, container widths, buttons, motion (section 4).
`navigation`	`{ menus: Record<name, NavItem[]> }`	Named menus (`primary`, `footer`, `legal`, `mobile`…) used by header and footer components.
`header`, `footer`	`Node`	Site chrome, one tree each; pages can hide or make them transparent through `layout`.
`globalSections`	`{ id, name, node }[]`	Reusable blocks; a page node with `ref` renders the shared tree.

`pages`	`PageDocument[]`	Every page: metadata plus `sections: Node[]`.
`forms`	`FormDefinition[]`	Field definitions, validation, spam settings and workflow actions (email, webhook, integrations).
`collections`	`CollectionDefinition[]`	CMS: field schema plus items; items are rendered by collection-template pages or bound into lists.
`redirects`	`Redirect[]`	Exact, wildcard (`*`) and splat (`:splat`) redirects with 301/302/307/308.
`customCss`	string ≤ 100 kB	Site-wide CSS, sanitised before output.

PageDocument

Field	Notes
`id`, `title`	Stable id; title ≤ 200 chars.
`slug`	Path without leading slash, lowercase segments (`about`, `blog/news`); `""` is the homepage.
`kind`	`page` · `collection-template` (rendered at `/{slug}/{item}` for a collection) · `not-found`.
`status`, `publishAt`	`draft` · `published` · `scheduled` · `archived`; scheduled pages go live at `publishAt`.
`seo`	Title, description, canonical, robots, Open Graph, JSON-LD.
`layout`	Header `default | transparent | hidden`, footer `default | hidden`.
`customCss`, `scripts`	Page CSS (≤ 50 kB) and head/body code (requires `code.manage` to change).
`translations`	Per-locale title, slug and SEO overrides.
`sections`	The page content: an ordered list of root `Node`s.

3. The node tree

All visible content, from a whole section down to a single button, is a `Node`. A node names a registered component by `type`, carries that component's `props`, and may have `children`. A page is therefore a tree, and the whole site is a forest: header, footer, every page and every global section.

Field	Meaning
`id`	Unique within the site; also the CSS hook `.a-n-<id>`.
`type`	Registered component (`section`, `heading`, `hero`…).
`variant`	Optional component variant (layout preset).
`props`	Content and settings, validated by the component's Zod schema.
`style`	Responsive style overrides (section 4).
`animation`	Preset, trigger (load/scroll), duration, delay, easing.
`visibility`	Hide per breakpoint, schedule, or rule.
`binding`	Bind a CMS collection into a list prop.
`children`	`Node[]` (recursive).
`ref`	Points at a global section instead of inline content.
`customCss`, `anchor`	Node-scoped CSS; in-page link target.
`locked`, `hidden`, `name`	Editor-only conveniences.

```
{
  "id": "home-11",
  "type": "section",
  "props": {
    "tone": "default",
    "containerWidth": "default",
    "spacingTop": "lg"
  },
  "style": { "padding": { "base": "96px
0",
                          "mobile": "56px
0" } },
  "animation": { "preset": "fade-up",
                 "trigger": "scroll" },
  "children": [
    { "id": "home-10", "type":
"container",
      "props": { "width": "auto" },
      "children": [
        { "id": "home-8", "type":
"heading",
          "props": { "text": "Leela
Garden",
                     "level": 1 } }
      ] }
  ]
}
```

A section from the Leela Garden bundle (trimmed, style and animation added for illustration).

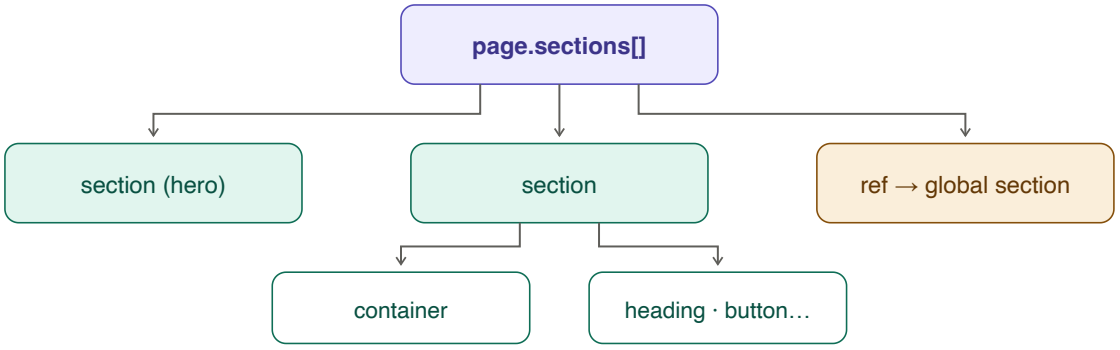


Figure 3. A page is a tree of nodes; a node with `ref` renders a shared global section.

4. Styles, breakpoints and theme tokens

Responsive style values

Every style key (spacing, sizing, layout, flex/grid, text alignment, background, colour, border, radius, shadow, typography overrides and more) accepts either a plain value or a responsive object:

```
type Responsive<T> = T | { base?: T; laptop?: T; tablet?: T; mobile?: T }
// laptop ≤ 1279px · tablet ≤ 991px · mobile ≤ 767px (desktop-first: base applies
everywhere)
```

When rendering, `compileTreeCss` (`packages/design-tokens`) walks every tree on the page and emits one rule per node, scoped as `.atrium-site .a-n-<id>`, plus `@media (max-width: …)` blocks for the breakpoint overrides. CSS values are checked by `isSafeCssValue`: no `url(javascript:…)`, no `expression()`, no breaking out of the declaration. Strict schemas reject unsafe values on input; lenient schemas drop them when reading stored data.

Theme tokens → CSS custom properties

Group	Contents	Becomes
`colors`	primary, primaryContrast, secondary, accent, background, surface, surfaceAlt, text, muted, border, inverse, inverseText, success, warning, danger (+ custom)	`--a-color-primary`, `--a-color-text` …
`typography`	heading/body font families (Google, local, custom, system), base size 12–22, modular scale 1.05–1.8, weights, line heights, letter spacing	`--a-font-heading`, `--a-text-xs … -6xl`: fluid sizes using `clamp()` between a mobile and a desktop scale
`spacing`	unit, section rhythm (compact · regular · airy)	section padding variables
`radius`, `shadows`, `borders`	sm/md/lg/xl/full, sm/md/lg, width/style	`--a-radius-md`, `--a-shadow-lg`
`containerWidths`	narrow, default, wide, full	container max widths
`buttons`, `motion`	button shape/style; motion intensity and easing	button variables; animation runtime config

Components never hard-code colours. They use tokens, plus local "tone" variables (`--t-bg`, `--t-text`, `--t-btn-bg` …) that sections switch between (`default`, `surface`, `inverse`, `on-media`). So changing the theme restyles every page without touching content.

5. The component registry

`packages/component-registry` defines what a node `type` means. It currently registers **65 components**: 2 site-level (`site-header`, `site-footer`), 37 sections, 12 blocks and 14 elements. Each `ComponentDefinition` has:

Field	Purpose
`type`, `name`, `category`, `level`	Identity and where it may be placed (site, section, block, element).
`props`	A Zod object schema: the single source of truth for that component's props, defaults and validation.
`defaultProps`, `variants`, `defaultVariant`	Starting content and layout presets for the element picker and the AI.
`controls`, `styleControls`	What the inspector shows (text, media, list, select…) and which style keys are editable.
`children`	Whether it accepts children, which types, min/max.
`bindable`	Which list prop a CMS collection can feed, and the item fields.
`animations`, `ai`, `tags`	Recommended animation presets; guidance for the AI agent (when to use, example).

The registry is shared. The editor uses it for the picker and inspector, `builder-core` uses it to validate and fill props, the AI agent reads its descriptions, and `renderer-core` maps each `type` to a React renderer.

6. Storage in PostgreSQL

Drafts are stored in pieces, so two people editing different pages don't conflict. Published versions are stored whole, so serving needs a single row.

Table	Holds	Notes
`websites`	Identity, slug, status, deployment mode, `published_version_id`, dashboard branding	Points at the live version.
`website_settings`	`settings`, `navigation`, `header`, `footer` (JSON), `custom_css`	Own `revision` counter.
`themes`	`tokens` (JSON)	One theme per website.
`pages`	Page metadata columns + `document` (JSON: the sections tree)	Own `revision`; one row per page.
`page_versions`	Per-page history (`document` JSON)	Autosave history, restore.
`forms`, `collections`, `global_sections`, `redirects`	The remaining parts of the definition	Plus submissions, items, etc.
`website_versions`	`snapshot` (the complete `SiteSnapshot` JSON), `number`, `checksum`, `message`	Immutable; one per publish.
`media`	Metadata (key, type, size, dimensions, variants, checksum)	Bytes live in storage, not the DB.

`loadDraft()` in `packages/site-store` reassembles these rows into one `WebsiteDefinition`. The editor, preview, export and publish all work from that one assembled value.

---

7. Parsing and validation

Every way content enters the system goes through the same funnel. Loose input is accepted, normalised against the registry, then checked against the strict schema before anything is stored.

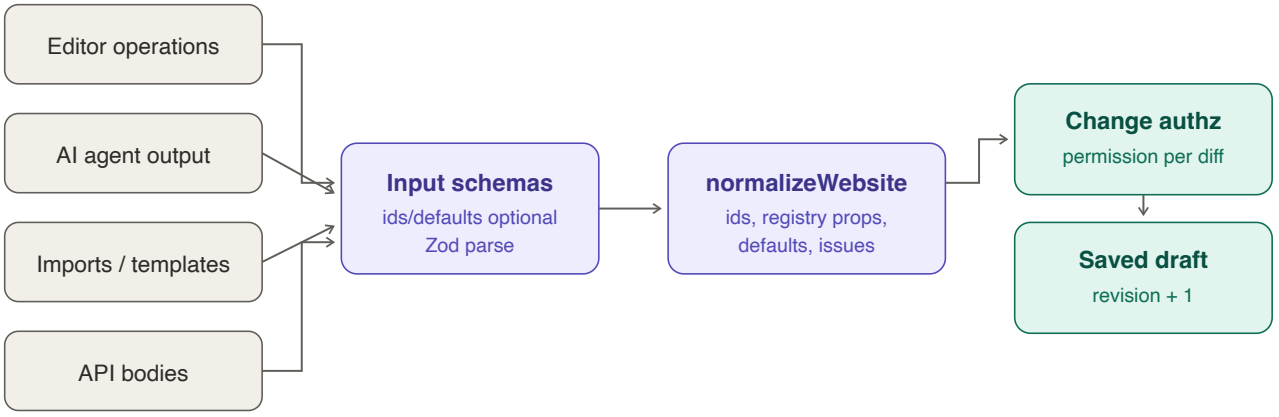


Figure 4. One validation funnel for every source of content.

1. **Input schemas** (`WebsiteInputSchema`, `PageInputSchema`, `NodeInputSchema`) accept partial data: ids and defaults are optional, so the AI or an import can send just the content.
2. **`normalizeWebsite` / `normalizePage`** (`builder-core`):
   - assign missing or duplicate ids and keep exactly one homepage, de-duplicating slugs;
   - look up each node's `type` in the registry and parse its `props` with that component's Zod schema, filling defaults and coercing types;
   - enforce the allowed children;
   - collect `NodeIssue`s (`error`/`warning` with a path) instead of failing on the first problem. Unknown types are kept and rendered as a visible placeholder, never silently dropped.
3. **Strict vs lenient schemas.** `StrictStyleSchema` and `StrictThemeTokensSchema` reject unsafe CSS or contrast-breaking themes on input with a message. The lenient `StyleSchema` and `ThemeTokensSchema` drop such values when reading stored data, so a stored document can always render.
4. **Change authorization** (`siteChangeRequirements`, `pageChangeRequirements`). The API diffs the old and new definition and requires a permission per kind of change:
   - scripts, custom CSS and embeds need `code.manage`;
   - the theme needs `theme.update`;
   - publishing or scheduling needs `page.publish`;
   - menus, redirects, forms, collections and SEO each have their own permission.

   A member who may edit text therefore cannot inject a script through the same save.
5. **Persist with optimistic concurrency**: each save names the `revision` it was based on; a stale revision is rejected (409), so concurrent editors never overwrite each other silently.

8. Editing: operations and revisions

The visual editor does not send whole documents. It sends small **operations**, applied by `applyOperation` in `builder-core` on the client (for instant feedback) and again on the server (as the authority):

```
addNode · updateNode · setProp · setStyle · replaceNode · moveNode · removeNode ·
duplicateNode
createPage · updatePage · deletePage · duplicatePage
updateTheme · updateSettings · setMenu · createForm · updateForm · deleteForm ·
setRedirects · setCustomCss
```

Each operation is validated (for example, `setStyle` goes through the strict style schema and persists only the sanitised result), checked against the change permissions, and recorded for undo and redo. Page saves bump that page's `revision` and append to `page_versions`. Site-level saves bump `website_settings.revision`.

9. Publishing: freezing a snapshot

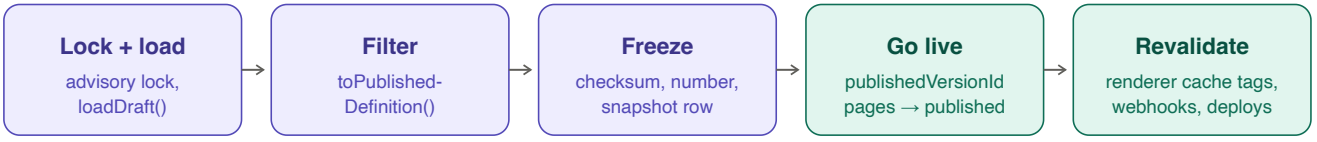


Figure 5. `freezeVersion()` in `packages/site-store/src/publish.ts`, run inside one database transaction.

1. **Lock and load.** `pg_advisory_xact_lock(hashtext(websiteId))` serialises publishes of the same site; `loadDraft()` assembles the definition.
2. **Filter** with `toPublishedDefinition()`:
   - archived pages and pages scheduled for the future are dropped;
   - everything else is marked `published`;
   - only published collection items and enabled redirects are kept;
   - the result is re-parsed with `WebsiteDefinitionSchema`.
3. **Freeze.** Take a SHA-256 checksum of the published definition, assign the next version number, and build the `SiteSnapshot` (with `apiBaseUrl`, primary domain and runtime keys) through `SiteSnapshotSchema.parse`. Store it in `website_versions.snapshot`.
4. **Go live**: set `websites.published_version_id` and `status = PUBLISHED`, and mark the included pages published. Rolling back just points `published_version_id` at an older version.
5. **After commit.** The API calls `POST {RENDERER_URL}/api/revalidate` with the shared secret; the renderer runs `revalidateTag` for that site's cache tags. It also emits the `website.published` event for webhooks and starts an external deployment if the site uses "independent" mode.

> Drafts and the live site can never mix: the renderer reads only `website_versions.snapshot` through `published_version_id`. Preview links (`/_preview/<token>`) are the only path that renders drafts, and they are never cached.

10. Serving a request

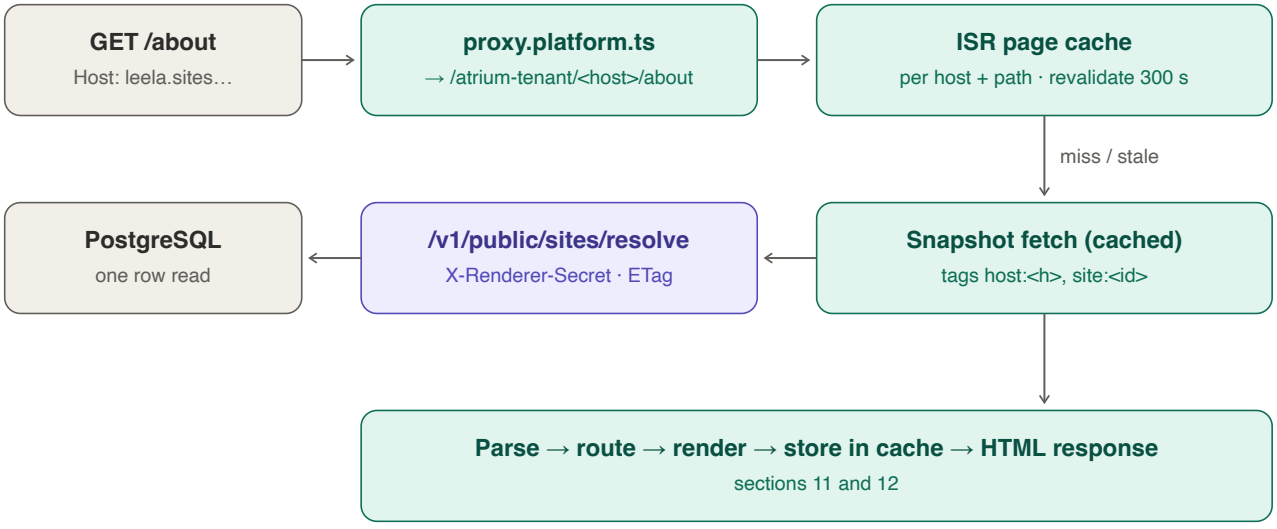


Figure 6. A visitor request. On a cache hit the response is served from the first teal box and nothing else runs.

1. **Edge.** nginx (or Caddy) terminates TLS and forwards `{sites domain}`, `*.{sites domain}` and verified custom domains to the renderer on port 3001, keeping the `Host` header.
2. **Host rewrite.** `proxy.platform.ts` rewrites the request to an internal route:
   - `/atrium-tenant/<host>/<path>` for live sites;
   - `/_site/<slug>` for local development;
   - `/_preview/<token>` for previews.

   Each host and path pair becomes its own cache key. Requests that try to reach the internal `/atrium-*` routes directly are blocked.
3. **Page cache (ISR).** Tenant pages use `revalidate = 300`: a rendered page is served from cache and refreshed in the background after five minutes, or immediately when publishing revalidates its tags.
4. **Snapshot fetch.** On a miss, `getSnapshotForHost()` calls `GET /v1/public/sites/resolve?host=…` with `X-Renderer-Secret`. The fetch is itself cached and tagged `host:<h>` and an all-sites tag, so publish can purge it.
5. **Host → website** (API, `PublicService.websiteForHost`):
   - `{slug}.{PLATFORM_SITES_DOMAIN}` maps to a website by its slug;
   - otherwise the host is looked up in `domains`, and only verified domains (TXT proof) that point at the platform are served;
   - suspended or unpublished sites return 404.

   The response is the stored snapshot, with an ETag (`checksum-version`), so revalidation can be a cheap 304.

11. Rendering a page

Parse and route

- `coerceSnapshot()` re-validates the JSON with `SiteSnapshotSchema.parse`. The renderer never trusts the shape, even from its own API.
- `resolveRequest()` → `resolvePageForPath()` (`renderer-core/routing.ts`) handles the following, in order:
  1. enabled redirects (exact, wildcard, `:splat`), which return a redirect response;
  2. the locale prefix, according to the site's localization strategy;
  3. page slugs, including translated slugs;
  4. collection-template pages, where `/{slug}/{item}` binds the CMS item into scope;
  5. otherwise the site's 404 page (or a generated one), served with status 404 and `noindex`.
- A **`RenderContext`** is built for the page:
  - the definition, theme and registry;
  - the global-sections map;
  - the template scope (`vars`, `site`, `page`, the CMS `item`);
  - locale, mode (`live` / `preview` / `editor`), base path and API base URL.

Build the CSS (`getSiteCss`)

Each page carries one `<style>` block, assembled in this order:

1. custom `@font-face` rules, plus the Google Fonts stylesheet with preconnect;
2. `themeToCss(theme, '.atrium-site')`: all tokens as CSS custom properties;
3. component CSS, with only the chunks for components used on this page (shared CSS served as `/atrium-assets/renderer.css`);
4. the animation base CSS, which hides not-yet-animated elements to avoid a flash;
5. `compileTreeCss`: per-node `.a-n-<id>` rules with breakpoint media queries;
6. the site `customCss`, then the page `customCss`, both through `sanitizeCss`.

The whole block is escaped so no value can close the `<style>` element.

Render the tree (`NodeView`)

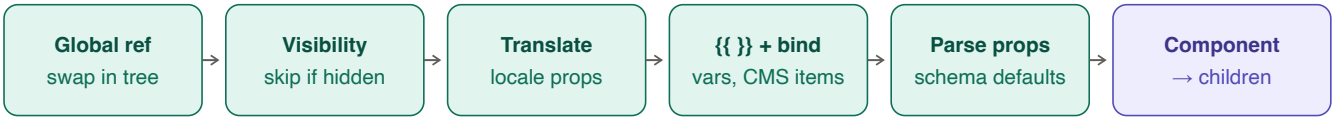


Figure 7. What `NodeView` does for every node before handing it to the component renderer.

1. **Global sections**: a node with `ref` is replaced by the shared tree (wrapped as `a-global`).
2. **Visibility**: hidden nodes (by rule, schedule or flag) are skipped; per-breakpoint hiding is done in CSS.

3. **Translations**: per-locale prop overrides are applied.
4. **Interpolation and binding**:
   - `{{vars.phone}}`, `{{item.title}}` and similar are resolved from the scope (HTML-escaped for HTML props);
   - a `binding` fills a list prop from a CMS collection.
5. **Prop parsing**: `parseWithSchema` applies the registry schema again, so missing props get defaults and bad values can't crash a render.
6. **Component**:
   - the renderer registered for `type` outputs semantic HTML with the class `a-n-<id>`, the animation attributes (`data-a-anim`) and the node's children (through `NodeView` again);
   - an unknown type renders a placeholder (`a-unknown`) instead of failing the page.

The document shell (`SiteDocument`) adds the metadata built by `buildMetadata`:

- title and description;
- canonical URL and hreflang alternates;
- robots, Open Graph and Twitter cards;
- icons;
- JSON-LD (organization, local business, page schema).

It also adds the scripts allowed by consent, the cookie banner, the analytics beacon and the client islands.

---

12. In the browser

- **HTML first.** The page is complete, server-rendered HTML (good for SEO and first paint). Only small client **islands** hydrate:
  - forms (validation, captcha, submit to `POST /v1/public/forms/:formId/submit`);
  - booking, carousel, lightbox, crossfade and map facades;
  - header behaviour;
  - `SiteRuntime`.
- **SiteRuntime** lazy-loads GSAP only on pages with animations, runs `initAnimations` over `[data-a-anim]` elements, and manages cookie consent, consent-gated scripts and the cookieless analytics beacon.
- **Client navigation** (`ClientLinks`):
  - same-origin link clicks become Next.js client navigations, which fetch only the next page's server payload (`?_rsc=`), not a full document;
  - the clicked link dims at once, and a slim loading bar in the site's primary colour appears after 120 ms; after 12 s it falls back to a normal page load.
- **Browser cache** (Settings → Page loading, `settings.performance.cachePages`):
  - on: links are preloaded on hover/focus/touch and header links when idle; visited and preloaded pages stay in the router cache (`staleTimes.static` = 24 h) until the visitor reloads;
  - off: nothing is preloaded and every click fetches the latest page.

13. Site bundles (export / import)

A website can be exported as a zip and imported elsewhere. The layout mirrors the definition:

```
site.json        settings, navigation, header, footer, globalSections, redirects,
customCss, schemaVersion, bundle
theme.json       theme tokens
pages/*.json     one PageDocument per file (_home.json = homepage)
forms/*.json     form definitions
assets/<path>    optional media files, referenced in any JSON as "asset:<path>"
```

Import works as follows:

1. The zip is unpacked with limits: 300 MB upload, 400 MB unpacked, 2,000 entries, at most 500 referenced assets.
2. Every `asset:` reference is uploaded through `MediaService`, which detects the real file type, sanitises SVGs, checks storage quotas and reuses files already in the library.
3. The references are rewritten to media URLs: `src` objects also get `mediaId`, width, height and type, so the renderer can serve responsive image variants.
4. The rewritten bundle goes through the normal validation funnel (section 7) and a new website is created. Nothing existing is overwritten.

14. Security at the boundaries

Boundary	Protection
Content input	Strict Zod schemas; per-diff permissions (scripts, CSS, embeds need `code.manage`); revision checks.
CSS	`isSafeCssValue` for style values, `sanitizeCss` for custom CSS, `escapeCssForHtml` for the `<style>` block, node rules scoped to `.atrium-site`.
HTML and text	React escaping by default; `sanitizeHtml` for rich text; HTML-escaped interpolation for HTML props.
URLs and media	Safe-URL schemas (no `javascript:`); uploads type-sniffed from bytes, SVGs sanitised, user files served with a sandboxing CSP from the API/media origin, never the admin origin.
Renderer ↔ API	Snapshot and preview endpoints require `X-Renderer-Secret`; the renderer re-validates every snapshot; revalidation needs the same secret.
Domains	Only verified domains are served; on-demand TLS asks the API before issuing certificates.
Tenancy	Every resource is resolved to its organization and website before access; website-scoped members (e.g. Website Manager) are limited centrally in `AccessService`.

15. Source map

Where	What
`packages/schema/src/{website,page,node,style,theme,settings}.ts`	The data model (Zod schemas and types)
`packages/schema/src/responsive.ts`	Breakpoints and responsive values
`packages/component-registry/src`	65 component definitions and their prop schemas
`packages/builder-core/src/{normalize,operations,change-authz}.ts`	Normalisation, editor operations, change permissions
`packages/site-store/src/{draft,publish}.ts`	`loadDraft`, `toPublishedDefinition`, `freezeVersion`
`apps/api/src/modules/public/`	`/public/sites/resolve`, preview, forms, analytics
`apps/api/src/modules/publishing/`	Publish, rollback, renderer revalidation
`apps/renderer/proxy.platform.ts`	Host → internal route rewrite
`apps/renderer/lib/{snapshot,platform,site,client-links}.tsx`	Snapshot fetch and cache, document shell, client navigation
`packages/renderer-core/src/{routing,node-view,props,styles}.ts(x)`	Routing, node pipeline, interpolation, CSS assembly
`packages/design-tokens/src/{tokens,styles}.ts`	`themeToCss` (tokens → CSS variables, fluid type scale), `compileTreeCss` (per-node rules)
`packages/animations/src`	Animation presets and runtime